DSM A2 Part 1 Final Report: Yelp Open Dataset

Shivansh Verma

March 30, 2026

Contents

1 Data Acquisition and MongoDB Schema Design **2**

2 Entity-Relationship (E-R) Diagram (MongoDB) **5**

3 Document Schema Diagram (MongoDB Collections) **6**

4 Neo4j Property Graph Model **6**

1 Data Acquisition and MongoDB Schema Design

1. Core ETL Logic & Data Loading

The extraction, transformation, and loading (ETL) pipeline was developed in Python utilizing the `pymongo` library. Instead of a direct import mirroring the exact JSON constraints, we apply specific transformations defined in `etl/load_yelp_mongo.py`. The pipeline streams large `.json` files in batches, converts date strings into MongoDB-compatible Date values, restructures `checkin.date` into a `dates` array, normalizes user `friends` and `elite` into arrays, and maps Yelp IDs to MongoDB `_id` where appropriate.

The script runs sequentially via:

```
python3 etl/load_yelp_mongo.py --data-dir ../data
```

2. Collection Structure and Inter-relations

The schema was modeled strictly based on the relational and analytical needs of the provided dataset parameters. Instead of mimicking the six source JSON files 1:1, the design separates bounded attributes (embedded) from unbounded elements (referenced) to prevent hitting MongoDB's 16MB document size limit as relationships scale.

Below is an overview of the five primary collections, their respective roles, document identifiers, and referential structures.

Businesses Collection

Purpose: Acts as the central anchor for the database, housing core attributes, categorizations, and static metadata for various commercial locations.
Identifier Field: `_id` (String, mapped directly from Yelp's native `business_id`).
Linking Relationships: Is the parent entity. Other collections (like reviews, tips, and checkins) reference a business by its `_id`.
Key Fields: `name`, `city`, `stars`, `review_count`, `is_open`. Attributes like `categories` (Array) and `hours` (Object) are strictly bound to the business and do not continuously grow, making them ideal to embed directly for localized queries.

Users Collection

Purpose: Stores user metadata, engagement metrics, and social network ties.
Identifier Field: `_id` (String, mapped directly from Yelp's native `user_id`).
Linking Relationships: The `friends` field contains an array of strings referencing other `_id` fields within the same collection (N:M self-referential). It is also globally referenced by the Reviews and Tips collections.
Key Fields: `name`, `yelping_since` (Date), `elite` (Array), `average_stars`, and various metrics. Due to the scale of social connectivity, `friends` arrays are populated strictly via references rather than embedded documents to avoid extreme growth amplification.

Reviews Collection

Purpose: Logs independent evaluations made by users on specific businesses.
Identifier Field: `_id` (String, mapped from `review_id`).
Linking Relationships: Contains `user_id` and `business_id` fields, which inherently reference the Users and Businesses collections (N:1 mapping to both).
Key Fields: `stars`, `date` (Date object), `text`, `useful`. Referencing is strictly necessary here. If reviews were embedded within a user or business, heavily trafficked locations could easily crash string limitations.

Checkins Collection

Purpose: Tracks longitudinal visitation trends mapped over time.
Identifier Field: `_id` (String, mapped directly from `business_id`), forming a rigid 1:1 schema extension with the Business entity.

Linking Relationships: Because the `_id` maps to `business_id`, it explicitly references the Businesses collection.
Key Fields: `dates` (Array of Date objects). Although checkins are technically bounded to a business, separating them into a new collection prevents the core business documents from massive index bloat generated by extremely large, actively expanding sub-arrays of timestamps.

Tips Collection

Purpose: Captures smaller, qualitative offhand recommendations distinct from formal reviews.
Identifier Field: `_id` (Auto-generated assigned by MongoDB).
Linking Relationships: Utilizes distinct `user_id` and `business_id` fields to bridge across the Users and Businesses collections.
Key Fields: `text`, `date` (Date object), `compliment_count`. Like reviews, tips are unconstrained and inherently must maintain a normalized cross-reference.

3. Schema Design Justifications (Embedding vs. Referencing)

(a) Relationship Design Choices

- **Business → Attributes, Categories, Hours (Embedded):** These entities inherently map 1:1 to a specific business. Their data volume is strictly bounded (e.g., there are only 7 days in a week for hours, and a finite set of attributes). Embedding these properties prevents expensive cross-collection joins just to retrieve basic business profile information.

- **User → Friends (Referenced):** The Yelp social graph is massive and highly interconnected (N:M). Embedding a user's friends list as full documents would exponentially duplicate data. Even embedding an array of `user_id` strings risks exceeding MongoDB's 16MB document size limit for highly active "elite" users. Thus, referencing is strictly required.

- **Business → Reviews & Checkins (Referenced):** Yelp reviews and checkins grow unboundedly over time (1:N mapping). Embedding thousands of reviews directly inside a single `business` document would inevitably breach the 16MB limit, severely degrading read/write performance long before failing entirely. Keeping these in standalone collections referencing `business_id` is the only scalable approach.

- **User → Reviews & Tips (Referenced):** Similar to businesses, a highly active user can generate an unbounded number of reviews and tips over their tenure. These must uniquely exist in their own collections, maintaining references back to the contributing `user_id`.

(b) Read and Write Trade-offs

Embedding Trade-offs (Business Attributes/Categories/Hours):

- **Read Advantage:** Yields unmatched read performance. A single query to the Businesses collection retrieves the entire business profile, navigating latency from `$lookup` joins.

- **Write Disadvantage:** Granular updates (like changing a single attribute) require rewriting the entire embedded object mapping. However, since business static attributes rarely change, this write penalty is highly negligible.

Referencing Trade-offs (Reviews/Checkins/Friends):

- **Write Advantage:** Excellent write performance and infinite scalability. When a new review is posted, it simply inserts a new document in the Reviews collection without ever locking or modifying the massive parent Business or User document.

- **Read Disadvantage:** Requires application-level joins or aggregation pipeline `$lookups`. Fetching a business alongside all its reviews demands querying multiple collections, increasing memory usage and retrieval latency. This trade-off is mitigated by strategic indexing.

(c) Indexing Strategy for Section 4 Queries

To efficiently execute the analytical queries mapped out in Section 4 of the assignment, the following targeted indexes were established inside the ETL loading script:

1. `db.businesses.createIndex({ "city": 1, "review_count": -1 })`
   - **Justification:** Queries 1 and 2 filter and aggregate by city while enforcing minimum review-volume thresholds. This index accelerates those filters and reduces scan volume before grouping.
2. `db.businesses.createIndex({ "categories": 1, "review_count": -1 })`
   - **Justification:** Queries 1, 2, 4, and 7 all touch category-level analysis with review-volume constraints. This multikey index supports category filtering and faster category aggregates.
3. `db.reviews.createIndex({ "business_id": 1, "date": 1 })`
   - **Justification:** Query 2 analyzes rating trends *over time*. This compound index supports extremely fast sequential retrievals natively sorted by date for any given business, removing the need for an expensive in-memory sort stage during the aggregation pipeline.
4. `db.reviews.createIndex({ "user_id": 1, "date": 1 })`
   - **Justification:** Queries 5 and 6 join users to review-level behavior. This compound index speeds user-centric traversals and supports optional temporal slicing.
5. `db.users.createIndex({ "yelping_since": 1 })`
   - **Justification:** Query 5 bins users by tenure on Yelp. This index helps tenure-range filtering and bucketing when user-side prefiltering is applied.
6. `db.checkins.createIndex({ "_id": 1 })` *(Auto-generated mapped from business_id)*
   - **Justification:** `checkins` use `business_id` directly as their explicit `_id`, granting them an automatic, uniquely optimal index at scale. For Query 7 (correlating check-in visit frequency with business star ratings), we natively rely on this `_id` index to flawlessly execute a 1:1 optimal pipeline join with the `businesses` collection on the same key.

2 Entity-Relationship (E-R) Diagram (MongoDB)

1. E-R Diagram

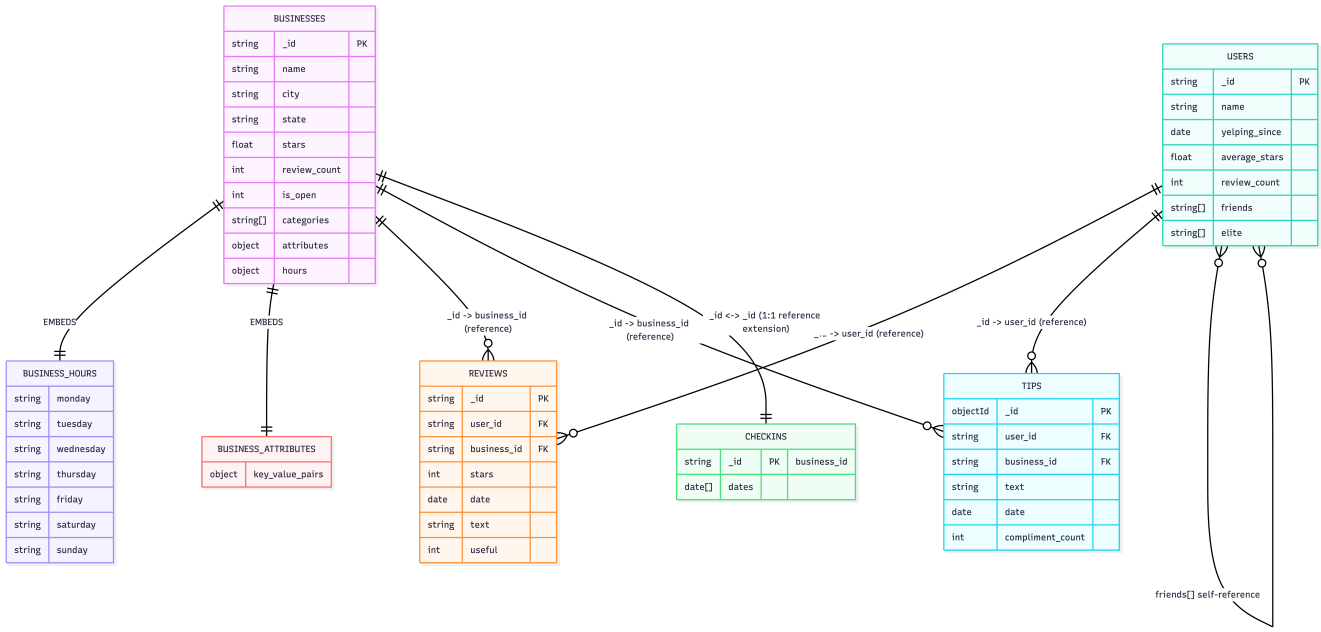

Figure 1: Entity-Relationship diagram for the MongoDB data model.

2. Explanation of Relationships and Attributes

- **Embedded relationships:** Business `hours` and `attributes` are embedded because they are tightly coupled with one business document and are queried together with business profile fields.
- **Referenced relationships:** Reviews and tips reference users and businesses using `user_id` and `business_id`, enabling unbounded growth without inflating parent documents.
- **1:1 extension:** Checkins uses `business_id` as `_id`, creating a direct one-to-one extension document for each business.
- **Self-reference:** User friendships are represented as an array of referenced user IDs (N:M style social links) rather than embedded user objects.
- **Key attributes retained in the diagram:** identifiers, rating fields, temporal fields, text payload fields, and array/object fields that drive the assignment queries.

3 Document Schema Diagram (MongoDB Collections)

1. Document Schema Diagram

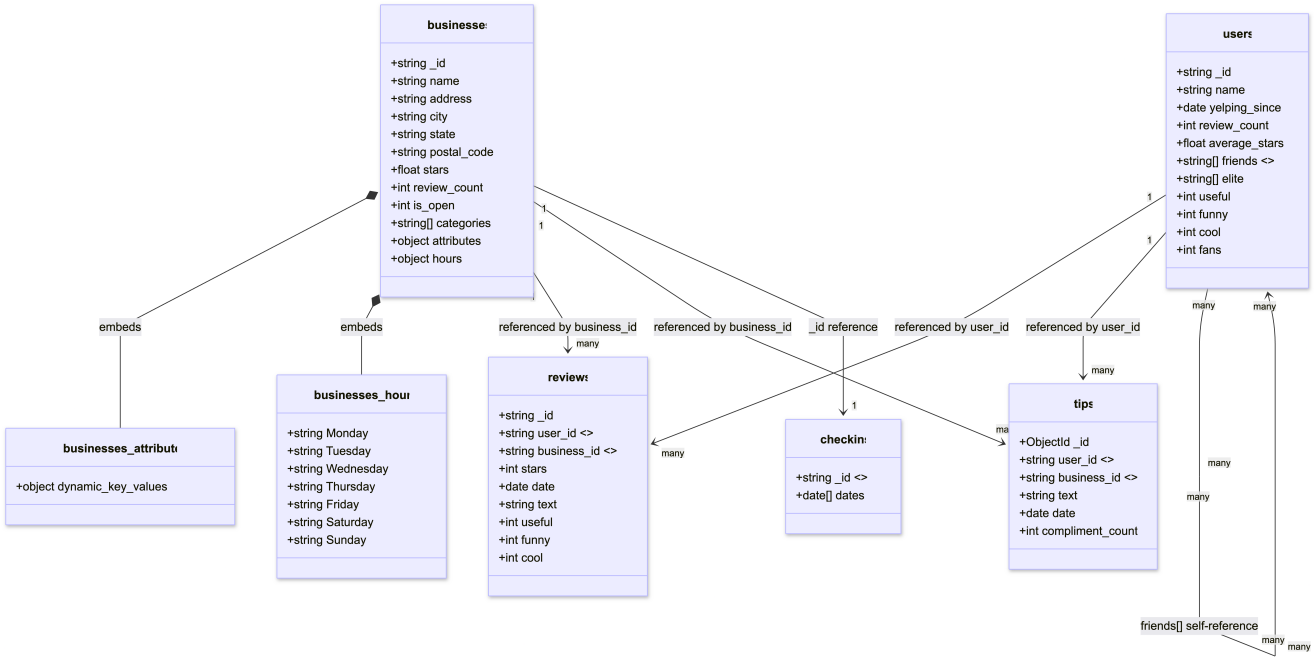

Figure 2: Document schema diagram with fields, types, nesting, and references.

2. Structural Design Justifications

- **Business document keeps bounded nested data embedded:** `hours` and `attributes` are naturally one-business scoped and usually read with the business profile.
- **High-volume activity is referenced:** `reviews` and `tips` are separate collections to avoid unbounded growth in user/business documents.
- **Checkin separation supports write scalability:** each check-in event appends to `checkins.dates` without repeatedly rewriting large business documents.
- **User social links use ID arrays:** storing `friends` as IDs avoids heavy duplication from embedded friend subdocuments and maintains compatibility with graph projection workflows.
- **ETL-consistent typing:** date fields are transformed to Date values in ETL, and categories/friends/elite are normalized into arrays for efficient aggregation and indexing.

4 Neo4j Property Graph Model

1. Graph Schema Design: Nodes & Properties

The Yelp dataset contains dense, multi-hop networking correlations (like Users having Friends who review Businesses in shared Categories). To maximize Cypher traversal performance, the property graph was modeled utilizing four primary Node representations:

- **(User) Node**
  Properties: `id` (String, Indexed UUID), `name` (String), `review_count` (Integer), `yelping_since` (Date Type), `average_stars` (Float).
- **(Business) Node**
  Properties: `id` (String, Indexed UUID), `name` (String), `city` (String), `state` (String), `stars` (Float), `review_count` (Integer).

- **(Review) Node**
  Properties: `id` (String, Indexed UUID), `stars` (Integer), `date` (Date Type), `text` (String), `useful` (Integer).

- **(Category) Node**
  Properties: `name` (String, Indexed, Unique). (Note: By extracting string arrays from JSON and upgrading them into explicit 'Category' nodes, we can trivially query "all users who reviewed businesses in X category" using simple path traversals).

2. Graph Schema Design: Relationships

- `(User)-[:FRIENDS_WITH]->(User)`
  Direction: Undirected (bi-directional implicit). Represents the N:M social web between users.

- `(User)-[:WROTE]->(Review)`
  Direction: Directed. Binds a user payload specifically to the rating document.

- `(Review)-[:REVIEWS]->(Business)`
  Direction: Directed. Assigns the sentiment payload explicitly to the localized entity.

- `(Business)-[:IN_CATEGORY]->(Category)`
  Direction: Directed. Defines the classification of a commercial entity (e.g., 'Restaurants', 'Active Life').

3. Property Graph Diagram

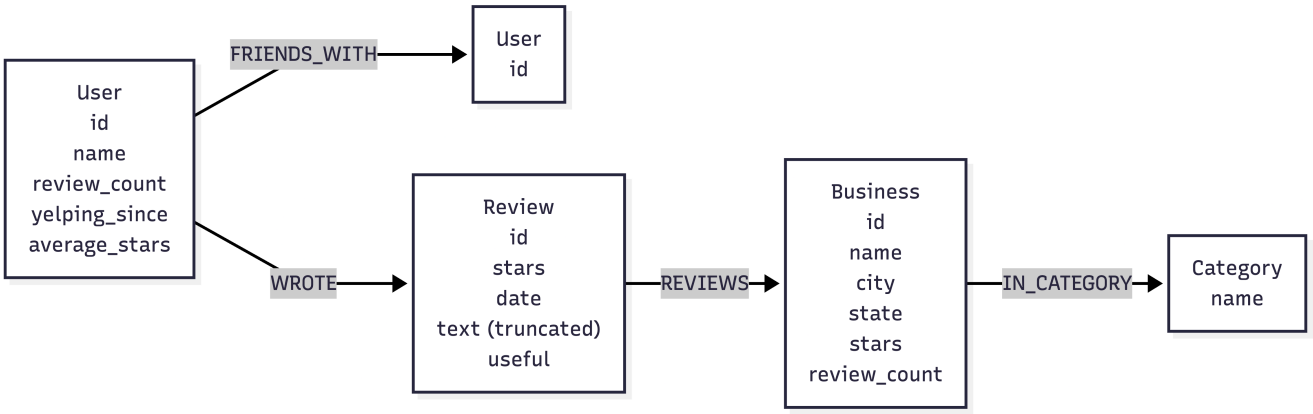

Figure 3: Neo4j property graph model (labels, directions, and key properties).

4. Modelling Choices & Handling Non-Graph Aspects

Treating Reviews as Nodes rather than Relationships:
Intuitively, a review appears functionally identical to an edge: `(User)-[:REVIEWED {stars: 5, date: '...'}]->(Business)`. However, directly injecting massive payloads (like paragraphs of raw string text and interaction metrics such as `useful` counts) into edge properties forces the database to evaluate heavy string clusters during large-scale network traversals like Community Detection or PageRank pathing. By extracting the Review into its own definitive *Node*, it maintains a lighter relationship bridging `User` → `WROTE` → `Review` → `REVIEWS` → `Business`.

Omission of Deeply Nested Document Blocks:
Property Graphs strictly require relationships to be bound to flat, scalar nodes. Deeply nested associative JSON arrays (like a business's operational `hours` map spanning sub-dictionaries of days mapped to time windows, or intricate nested `attributes`) do not map naturally to nodes or edges without creating severe fragmentation (e.g., mapping a dedicated `Monday` node).

Because spatial/temporal routing queries based on store opening hours are notoriously better suited to Document stores (like MongoDB parsing embedded JSON), complex dictionary fields like `hours` were intentionally omitted from the graph translation, prioritizing exclusively the relational ties driving the analytical graph queries defined in Section V.